

Fundamentals of Deep Learning and Programming

Lecture 2: **Useful Libraries** - NumPy, Pandas, Matplotlib

Fall Semester 2026

Adjunct Professor: Donghoon Kang



Python

Useful Libraries

Using GPU

```
# device (use GPU if available)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

a = torch.tensor([1., 2.], device=device)
b = torch.tensor([1., 2.])
print(a)
print(b)
```

tensor([1., 2.], device='cuda:0')
tensor([1., 2.])

c02_pytorch10.py



CPU



GPU

What are **Multiple Workers** (CPU Cores) ?



In the context of computer science, **multiple workers** are simply **different processes** or **threads** that perform tasks simultaneously to improve efficiency. When we talk about multiple workers, we are often referring to the **CPU cores** in a computer.

A **CPU (Central Processing Unit)** is the "brain" of a computer. In older computers, a single CPU had only one core, which could handle only one task at a time. However, modern CPUs often have multiple cores (e.g., dual-core, quad-core, octa-core). Each core is essentially an independent processing unit.

- **Single-core CPU:** Can execute only one task at a time. If it needs to do two things, it has to switch between them very quickly, which can slow down the overall process.
- **Multi-core CPU:** Each core can work on a different task simultaneously. This allows the computer to perform **many tasks at the same time**, significantly boosting performance.

What are Multiple Workers (CPU Cores) ?



- In the context of data loading for machine learning, a single worker (or a single process) would load data sequentially.
- Using multiple workers means you're using the multiple cores of your CPU to load data in parallel.
- For example, if you have 8 CPU cores and set `num_workers=4`, four cores will be used by the `DataLoader subprocesses to load and preprocess data`, and the remaining four cores will be available for other tasks.
- This makes the data loading process much faster and prevents the CPU from becoming a bottleneck.

argparse (1)

: a **standard Python library** that handles **command-line arguments**.

```
import argparse

def main():
    # 1. Create parser
    parser = argparse.ArgumentParser(description="my simple code")

    # 2. Add argument for number of iterations
    parser.add_argument("-my_iterations", type=int, default=5,
                        help="Number of times to run the loop (default: 5)")

    # 3. Parse arguments
    args = parser.parse_args()

    # 4. Use iterations in a for loop
    for i in range(args.my_iterations):
        print(f"Iteration {i+1}")

if __name__ == "__main__":
    main()
```

c04_argparse.py

```
python c04_argparse.py
```

```
Iteration 1
Iteration 2
Iteration 3
Iteration 4
Iteration 5
```

```
python c04_argparse.py --my_iteration 3
```

```
Iteration 1
Iteration 2
Iteration 3
```

```
parser.add_argument(my_args, **my_options)
```

my_args

Positional argument → " my_iteration" (no dashes)

Optional argument → "-my_iteration" (with dashes)

my_options: **keyword arguments** that customize behavior.
(ex.) type, default, help

type

- **Converts** the input **into a specific type**.
- Example: type=int → converts "5" to integer 5

default

- **Value used if the argument isn't provided**.
- Example: default=10

help

- A short **description shown** when running python c04_argparse.py --help.

argparse (2)

```
import argparse

def main():
    # 1. Create parser
    parser = argparse.ArgumentParser(description="simple example")

    # 2. Add argument for number of iterations
    parser.add_argument("my_iterations", type=int, default=5,
                        help="Number of times to run the loop (default: 5)")

    # 3. Parse arguments
    args = parser.parse_args()

    # 4. Use iterations in a for loop
    for i in range(args.my_iterations):
        print(f"Iteration {i+1}")

if __name__ == "__main__":
    main()
```

`python c04_argparse.py`



```
usage: c04_argparse.py [-h] my_iterations
c04_argparse.py: error: the following arguments are required: my_iterations
```

`python c04_argparse.py --my_iteration 3`



```
usage: c04_argparse.py [-h] my_iterations
c04_argparse.py: error: unrecognized arguments: --my_iteration
```

`python c04_argparse.py 3`

```
Iteration 1
Iteration 2
Iteration 3
```

argparse (3)

```
import argparse

def main():
    # 1. Create parser
    parser = argparse.ArgumentParser(description="simple example")

    # 2. Add argument for number of iterations
    parser.add_argument("-my_iterations", type=int, default=5,
                        help="Number of times to run the loop (default: 5)")

    # 3. Parse arguments
    args = parser.parse_args()

    # 4. Use iterations in a for loop
    for i in range(args.my_iterations):
        print(f"Iteration {i+1}")

if __name__ == "__main__":
    main()
```

```
python c04_argparse.py -h
```

```
usage: c04_argparse.py [-h] my_iterations

For loop iteration example

positional arguments:
  my_iterations  Number of times to run the loop (default: 5)

options:
  -h, --help  show this help message and exit
```

argparse (3)

```
import argparse

def main():
    # 1. Create parser
    parser = argparse.ArgumentParser(description="simple example")

    # 2. Add argument for number of iterations
    parser.add_argument("-my_iterations", type=int, default=5,
                        help="Number of times to run the loop (default: 5)")

    # 3. Parse arguments
    args = parser.parse_args()

    # 4. Use iterations in a for loop
    for i in range(args.my_iterations):
        print(f"Iteration {i+1}")

if __name__ == "__main__":
    main()
```

Matrix

(1) Addition

$$\begin{bmatrix} 2 & 1 & 0 \\ 3 & 4 & 1 \end{bmatrix} + \begin{bmatrix} -1 & 1 & 3 \\ 0 & 0 & 7 \end{bmatrix} = \begin{bmatrix} 1 & 2 & 3 \\ 3 & 4 & 8 \end{bmatrix}.$$
$$\begin{bmatrix} 1 & 2 \end{bmatrix} + \begin{bmatrix} 0 & -1 \end{bmatrix} = \begin{bmatrix} 1 & 1 \end{bmatrix}.$$

(2) Scalar multiplication

$$3 \begin{bmatrix} 1 & -1 & 2 \\ 0 & 1 & 5 \\ 1 & 0 & 3 \end{bmatrix} = \begin{bmatrix} 3 & -3 & 6 \\ 0 & 3 & 15 \\ 3 & 0 & 9 \end{bmatrix}.$$

$$A = [a_{ij}]$$

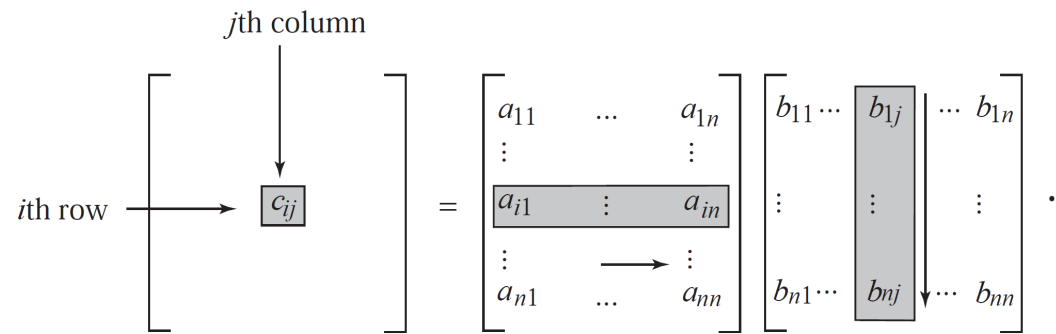
$$A = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix}.$$

$m \times n$ matrix

Next we turn to matrix multiplication. If $A = [a_{ij}]$, $B = [b_{ij}]$ are $n \times n$ matrices, then the product $AB = C$ has entries given by

$$c_{ij} = \sum_{k=1}^n a_{ik} b_{kj},$$

which is the dot product of the i th row of A and the j th column of B :



Similarly, we can multiply an $m \times n$ matrix (m rows, n columns) by an $n \times p$ matrix (n rows, p columns) to obtain an $m \times p$ matrix (m rows, p columns) by the same rule.

Example 1

$$A = \begin{bmatrix} 1 & 0 & 3 \\ 2 & 1 & 0 \\ 1 & 0 & 0 \end{bmatrix} \quad \text{and} \quad B = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 1 & 1 \end{bmatrix}.$$

Then

$$AB = \begin{bmatrix} 0 & 4 & 3 \\ 1 & 2 & 0 \\ 0 & 1 & 0 \end{bmatrix} \quad \text{and} \quad BA = \begin{bmatrix} 2 & 1 & 0 \\ 1 & 0 & 3 \\ 3 & 1 & 0 \end{bmatrix}.$$

Observe that $AB \neq BA$. ▲

Example 2

$$A = \begin{bmatrix} 2 & 0 & 1 \\ 1 & 1 & 2 \end{bmatrix} \quad \text{and} \quad B = \begin{bmatrix} 1 & 0 & 2 \\ 0 & 2 & 1 \\ 1 & 1 & 1 \end{bmatrix}.$$

$$AB = \begin{bmatrix} 3 & 1 & 5 \\ 3 & 4 & 5 \end{bmatrix}, \quad \text{and } BA \text{ is not defined.}$$

1 Column Convention in Mathematics

Traditional linear algebra often uses the **column convention**, where each input sample is represented as a column vector. In this case, the linear transformation is written as:

$$y = Wx + b \quad (1)$$

Here:

x	Input matrix of shape (D, B) , where D is the number of input features and B is the batch size
W	Weight matrix of shape (O, D) , where O is the number of output features
b	Bias vector of shape $(O, 1)$, broadcasted across all B samples
y	Output matrix of shape (O, B)

Thus, $x \in \mathbb{R}^{D \times B}$, $W \in \mathbb{R}^{O \times D}$, and $y \in \mathbb{R}^{O \times B}$.

Matrix Form: Column Vector Convention

In many mathematical treatments of neural networks, the linear transformation is expressed using column vectors:

$$y = Wx + b,$$

where

- x is an input column vector,
- W is the weight matrix,
- b is the bias vector.

This is the traditional notation used in linear algebra and theoretical descriptions of neural networks.

2 Row Convention in PyTorch

In contrast, PyTorch follows the **row convention**. Each row in a tensor represents one data sample, and each column represents a feature. For an input tensor $x \in \mathbb{R}^{B \times D}$, where B is the batch size and D is the number of input features, a linear layer is defined as:

$$y = xW^T + b \quad (2)$$

Here:

x	Input tensor of shape (B, D)
W	Weight matrix of shape (O, D)
b	Bias vector of shape (O)
y	Output tensor of shape (B, O)

Thus, $x \in \mathbb{R}^{B \times D}$, $W \in \mathbb{R}^{O \times D}$, and $y \in \mathbb{R}^{B \times O}$.

PyTorch Implementation: Row-Major Storage

PyTorch stores the weight matrix in a row-major layout for efficiency and consistency. In particular:

$$W \in \mathbb{R}^{(\text{out_features}) \times (\text{in_features})}.$$

Given a batch of input vectors arranged in a matrix:

$$x \in \mathbb{R}^{(\text{batch}) \times (\text{in_features})},$$

PyTorch performs the linear transformation as:

$$y = xW^T + b.$$

This ensures that:

$$x \in \mathbb{R}^{(B \times I)}, \quad W^T \in \mathbb{R}^{(I \times O)} \quad \Rightarrow \quad xW^T \in \mathbb{R}^{(B \times O)},$$

which matches the expected output shape.

Framework	Convention	Formula	Input Shape	Weight Shape	Output Shape
PyTorch / TensorFlow / NumPy	Row	$y = xW^T + b$	(B, D)	(O, D)	(B, O)
Math / MATLAB	Column	$y = Wx + b$	(D, B)	(O, D)	(O, B)

Table 3: Comparison of Row vs Column Conventions

Therefore, PyTorch uses the **row convention** by default. Each sample is a row vector, and the linear transformation in PyTorch is implemented as $y = xW^T + b$.

Although the mathematical form is commonly written as:

$$y = Wx + b,$$

PyTorch implements the computation as:

$$y = xW^T + b,$$

which is equivalent but adapted for efficient batch computation and memory layout.

NumPy

= Numerical Python

`conda install numpy`

`pip install numpy`



- **Multi-dimensional arrays (`ndarray`)** : Unlike Python lists, NumPy arrays store elements of the same type, allowing faster computation and less memory usage.
- **Mathematical operations**: A wide range of high-performance functions for linear algebra, statistics, and random number generation.
- **Vectorization**: Applying operations to **entire arrays at once**, making code more concise and much **faster**.
- **Easy Integration with other Python libraries** such as pandas, SciPy, scikit-learn, TensorFlow, and PyTorch.

performing **operations on entire arrays (vectors, matrices, tensors)** at once, instead of using explicit Python for loops.

```
import numpy as np

# Create arrays
a = np.array([1, 2, 3])
b = np.array([[1, 2, 3],
              [4, 5, 6]])

print(a)           # [1 2 3]
print(b)           # 2 rows, 3 cols

print(a.ndim)      # number of axes: 1
print(b.ndim)      # number of axes: 2
print(b.shape)     # (2, 3) -> 2 rows, 3 columns
print(b.dtype)     # data type (e.g., int64)

# Make arrays of zeros/ones or a range
z = np.zeros((2, 3))      # shape 2x3, all zeros (float)
o = np.ones((3, 2), dtype=int) # shape 3x2, all ones (int)
r = np.arange(0, 10, 2)   # [0 2 4 6 8]
lin = np.linspace(0, 1, 5) # [0.  0.25 0.5 0.75 1. ]
print(z, o, r, lin, sep="\n")
```

```
[1 2 3]
[[1 2 3]
 [4 5 6]]
1
2
(2, 3)
int64
```

```
[[0. 0. 0.]
 [0. 0. 0.]]
[[1 1]
 [1 1]
 [1 1]]
[0 2 4 6 8]
[0.  0.25 0.5 0.75 1. ]
```

c02_numpy01.py

```
B = [[10, 11, 12],  
      [20, 21, 22],  
      [30, 31, 32]]
```

list

```
print(B[0][1])
```

11



```
import numpy as np
```

```
A = np.array([[10, 11, 12],  
              [20, 21, 22],  
              [30, 31, 32]])
```

```
print(A[0, 1])    # row 0, col 1 -> 11  
print(A[1])       # entire row 1 -> [20 21 22]  
print(A[:, 2])    # entire col 2 -> [12 22 32]  
print(A[0:2, 1:3]) # rows 0..1, cols 1..2 -> [[11 12],[21 22]]
```

```
# Boolean indexing (filtering)
```

```
mask = (A % 20 == 0)    # True where divisible by 20  
print(mask)             # [[False False False], [ True False False], [ True False False]]  
print(A[mask])          # [20]
```

numpy

```
11  
[20 21 22]  
[12 22 32]  
[[11 12]  
 [21 22]]
```

```
[[False False False]  
 [ True False False]  
 [False False False]  
 [20]]
```

c02_numpy02.py

performing **operations on entire arrays (vectors, matrices, tensors) at once**, instead of using explicit Python for loops.



```
a = list(range(5))  
b = list(range(5))
```

list, for loop

```
result = [a[i] + b[i] for i in range(len(a))] # without vectorization
```

[0, 2, 4, 6, 8]

```
import numpy as np  
a = np.arange(5)  
b = np.arange(5)
```

numpy

```
result = a + b # vectorization
```

[0 2 4 6 8]

c02_numpy03.py



```
numbers = list(range(5))  
squared = [x**2 for x in numbers]  
print(squared)
```

list, for loop

[0, 1, 4, 9, 16]

```
numbers = np.arange(5)  
squared = numbers ** 2 # vectorization  
print(squared)
```

numpy

[0 1 4 9 16]

```
import numpy as np

A = np.array([[1, 2, 3],
              [4, 5, 6]]) # shape (2,3)

d = 10
print(A + d)
# [[11 12 13]
#  [14 15 16]]

print(type(d)) # <class 'int'>
print(type(A+d)) # <class 'numpy.ndarray'>
```

```
[[11 12 13]
 [14 15 16]]
```

```
<class 'int'>
<class 'numpy.ndarray'>
```

```
import numpy as np

A = np.array([[1, 2, 3],
              [4, 5, 6]]) # shape (2,3)
b = np.array([10, 20, 30]) # shape (3,)

print(A + b)
# [[11 22 33]
#  [14 25 36]]
```

```
[[11 22 33]
 [14 25 36]]
```

```
import numpy as np

A = np.array([[1, 2, 3],
              [4, 5, 6]]) # shape (2,3)
c = np.array([[100],
              [200]]) # shape (2,1)

print(A + c)
# [[101 102 103]
#  [204 205 206]]
```

```
[[101 102 103]
 [204 205 206]]
```

axis=0  go down columns (across rows)
axis=1  go across rows (across columns)

IMPORTANT



```
import numpy as np
```

```
B = np.array([[1, 2, 3],  
              [4, 5, 6]])
```

```
print(B.sum()) # 21
```

```
print(B.mean(), B.min(), B.max()) # 3.5 1 6
```

```
print(np.median(B), np.std(B)) # 3.5, standard deviation
```

```
# axis -----
```

```
print(B.sum(axis=0)) # column sums -> [5 7 9]
```

```
print(B.sum(axis=1)) # row sums -> [ 6 15]
```

```
# column-wise mean (axis=0)
```

```
col_means = B.mean(axis=0)
```

```
print("axis=0 (columns) mean:", col_means) # [2.5 3.5 4.5]
```

```
# row-wise mean (axis=1)
```

```
row_means = B.mean(axis=1)
```

```
print("axis=1 (rows) mean:", row_means) # [2. 5.]
```

21

3.5 1 6

3.5 1.707825127659933

[5 7 9]

[6 15]

axis=0 (columns) mean: [2.5 3.5 4.5]

axis=1 (rows) mean: [2. 5.]

c02_numpy05.py

reshape(), ravel()

```
import numpy as np

C = np.arange(12)           # [0..11]
print(C.reshape(3, 4))     # shape (3,4)
print(C.reshape(-1, 6))    # -1 lets NumPy infer -> (2,6)
print(C.ravel())            # flat view if possible
```

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
```

```
[[ 0  1  2  3  4  5]
 [ 6  7  8  9 10 11]]
```

```
[ 0  1  2  3  4  5  6  7  8  9 10 11]
```

c02_numpy06.py

`b = np.array([1., 2.])` is neither a row nor a column vector. It's a 1-D array with shape (2,). In NumPy, a vector's "direction" only becomes clear when you make it 2-D array.

```
import numpy as np

A = np.array([[1., 2.],
              [3., 4.]])
b = np.array([1., 2.])

row = b[None, :]          # shape: (1, 2) ← row vector
col = b[:, None]          # shape: (2, 1) ← column vector

# Equivalent forms
row2 = b.reshape(1, -1)  # (1, 2)
col2 = b.reshape(-1, 1)  # (2, 1)

# Matrix multiplication
print(b @ A)             # (2,) @ (2,2) → (2,) (behaves like a row in this context)
print(A @ b)             # (2,2) @ (2,) → (2,) (behaves like a column in this context)

print(row @ A)           # (1,2) @ (2,2) → (1,2)
print(A @ col)           # (2,2) @ (2,1) → (2,1)
```

`row = b[None, :]`

`None` (= `np.newaxis`) tells NumPy to 'add a new axis (dimension).'



```
[ 7. 10.]
[ 5. 11.]
```

```
[[ 7. 10.]]
```

```
[[ 5.]
 [11.]]
```

NumPy

np.stack()

np.stack() → adds a new axis.

axis=0 ↓ go down columns (across rows)
axis=1 → go across rows (across columns)

```
import numpy as np

X = np.array([1, 2, 3])
Y = np.array([4, 5, 6])

print(np.stack([X, Y], axis=0)) # [[1 2 3], [4 5 6]]
print(np.stack([X, Y], axis=1)) # [[1 4], [2 5], [3 6]]

print(np.vstack([X, Y])) # [[1 2 3], [4 5 6]]
print(np.hstack([X, Y])) # [1 2 3 4 5 6]
```

```
import numpy as np

X = np.array([1, 2, 3])
Y = np.array([4, 5, 6])

print(X.shape, Y.shape)

a = np.stack([X, Y], axis=0)
b = np.stack([X, Y], axis=1)
print(a.shape)
print(b.shape)
```

c02_numpy06.py

```
import numpy as np
```

```
A = np.array([[2., 1.],  
              [1., 3.]])  
b = np.array([1., 2.])
```

```
# Solve  $Ax = b$ 
```

```
x = np.linalg.solve(A, b)  
print("solution:", x)
```

```
solution: [0.2 0.6]
```

```
# Matrix multiply (dot product)
```

```
M = np.array([[1, 2],  
              [3, 4]])  
N = np.array([[10, 20],  
              [30, 40]])
```

```
print(M * N)          # caution: element-wise multiplication  
print(M @ N)          # matrix multiplication  
print(np.dot(M, N))   # matrix multiplication (M @ N)
```

```
print(np.linalg.eig(M)) # eigenvalues & eigenvectors
```

```
EigResult(eigenvalues=array([-0.37228132,  5.37228132]), eigenvectors=array([[ -0.82456484, -0.41597356],  
 [ 0.56576746, -0.90937671]]))
```

c02_numpy06.py

```
[[ 10  40]  
 [ 90 160]]
```

```
[[ 70 100]  
 [150 220]]
```

```
[[ 70 100]  
 [150 220]]
```

Pandas

= Spreadsheet using Python

`conda install pandas`

`pip install pandas`

- **Structured data**: Works perfectly with tabular data (rows & columns). Similar to Excel, but much more powerful.
- **Fast computations** with the flexibility of Python.
- Widely used in **data science**, **machine learning**, and finance.



Pandas

Series

A **1D** labeled **array**

- A Series is one-dimensional, but **pandas adds labels (index) to each value.**
- This makes the data more powerful and easier to use.

Series

A	10
B	20
C	30
D	40

index

value

```
import numpy as np
import pandas as pd

# Create a Series from a NumPy array
data = np.array([10, 20, 30, 40])
s = pd.Series(data, index=["A", "B", "C", "D"])
print(s)

print(s.values) # If you want to see values
```

```
A    10
B    20
C    30
D    40
dtype: int64
```

```
[10 20 30 40]
```

c02_pandas01.py

When you print a Series, pandas shows a summary of the object:

- **Index (labels)** → on the left (A, B, C, D)
- **Values** → on the right (10, 20, 30, 40)
- **dtype** → at the bottom (int64)

Pandas

DataFrame

A **2D** labeled table

Category	Age	weight
dog	3	15
cat	2	10
goat	8	35

```
import pandas as pd

# Create a DataFrame from a dictionary
data = {
    "Category": ["dog", "cat", "goat"],
    "Age": [3, 2, 8],
    "weight": [15, 10, 35]
}

#converts this dictionary into a DataFrame (a table)
df = pd.DataFrame(data)
print(df)
```

```
print(df["Age"])
```

```
print(df["Age"].mean()) # Average of "Age"
```

```
4.333333333333333
```

- **Index** → labels for rows (here: 0, 1, 2).
- **Series** → each individual column of the DataFrame.
- **DataFrame** → a collection of multiple Series sharing the same index.

```
Category  Age  weight
0      dog    3     15
1      cat    2     10
2     goat    8     35
```

```
0    3
1    2
2    8
Name: Age, dtype: int64
```

Pandas

Category	Age	weight
dog	3	15
cat	2	10
goat	8	35

```
import pandas as pd

# Create a DataFrame from a dictionary
data = {
    "Category": ["dog", "cat", "goat"],
    "Age": [3, 2, 8],
    "weight": [15, 10, 35]
}
df = pd.DataFrame(data)
```

```
print(df.iloc[0]) # first row
```

```
print(df.iloc[1]) # second row
```

```
print(df.iloc[:,0]) # first column
```

- **iloc** stands for integer-location based indexing

c02_pandas01.py

```
Category    dog
Age          3
weight      15
Name: 0, dtype: object
```

```
Category    cat
Age          2
weight      10
Name: 1, dtype: object
```

```
0    dog
1    cat
2   goat
Name: Category, dtype: object
```

Matplotlib

= making charts and plots

`conda install matplotlib`

`pip install matplotlib`

- You can make **line charts, bar charts, scatter plots, histograms, and more.**



Matplotlib

Draw a Line

```
import numpy as np
import matplotlib.pyplot as plt

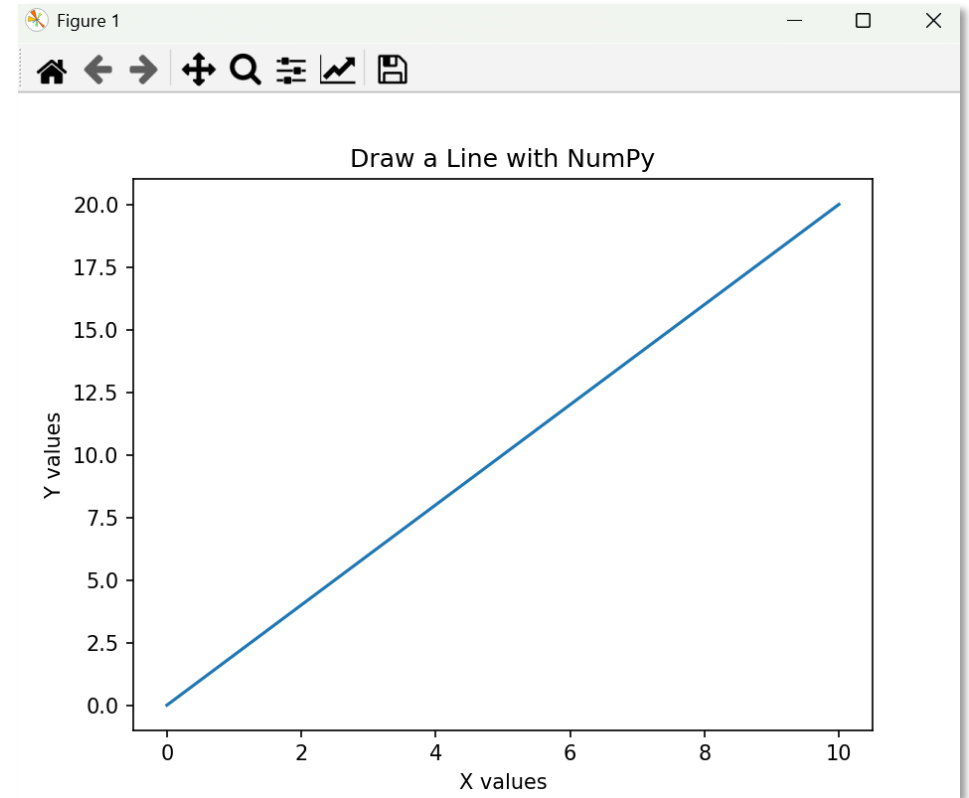
# Create data with NumPy
x = np.linspace(0, 10, 100)    # 100 points from 0 to 10
y = 2 * x                     # y = 2x

plt.plot(x, y)

plt.title("Draw a Line with NumPy")
plt.xlabel("X values")
plt.ylabel("Y values")

plt.show()
```

c02_matplotlib01.py



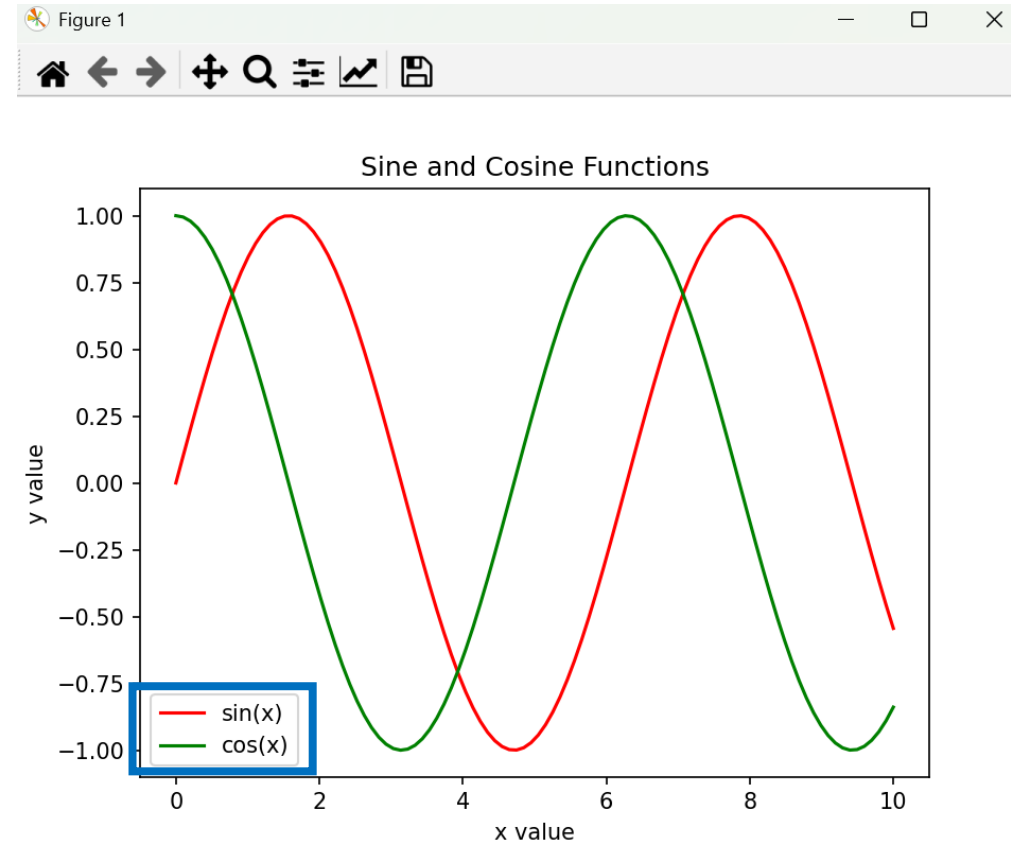
```
import numpy as np
import matplotlib.pyplot as plt

# Create data
x = np.linspace(0, 10, 100)
y1 = np.sin(x)
y2 = np.cos(x)

# Plot both
plt.plot(x, y1, label="sin(x)", color="red")
plt.plot(x, y2, label="cos(x)", color="green")

plt.xlabel("x value")
plt.ylabel("y value")
plt.title("Sine and Cosine Functions")
plt.legend()
plt.show()
```

c02_matplotlib02.py



Matplotlib

Multiple Graphs Side by Side

```
import numpy as np
import matplotlib.pyplot as plt

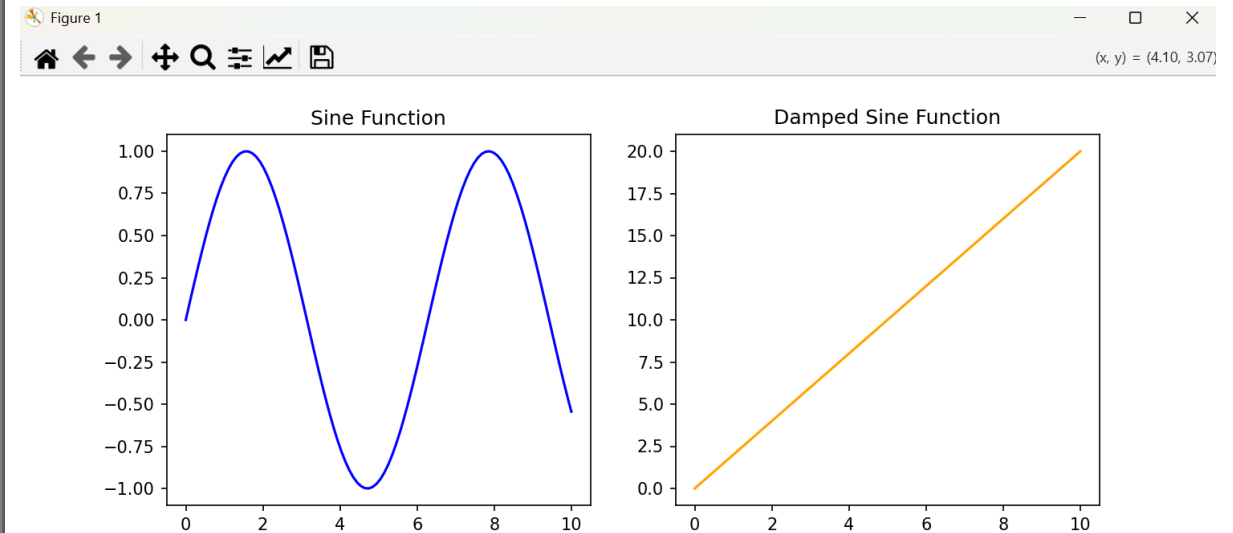
x = np.linspace(0, 10, 100)
y1 = np.sin(x)
y2 = 2 * x

# fig = whole figure (canvas),
# axes = subplots (small plots inside the figure)
# 1 row, 2 columns → two plots side by side
# figsize=(10, 4) → width=10 inches, height=4 inches
fig, axes = plt.subplots(1, 2, figsize=(10, 4))

# Left subplot
axes[0].plot(x, y1, color="blue")
axes[0].set_title("Sine Function")
axes[0].set_xlabel("x")
axes[0].set_ylabel("sin(x)")

# Right subplot
axes[1].plot(x, y2, color="red")
axes[1].set_title("Linear Function")
axes[1].set_xlabel("x")
axes[1].set_ylabel("2*x")

plt.tight_layout() # auto-adjust spacing between
                  # subplots
plt.show()
```



Matplotlib

```
import matplotlib.pyplot as plt
import numpy as np
```

Sample data

```
x = np.arange(5)
y = np.random.randint(1, 10, size=5)
scatter_x = np.random.rand(50)
scatter_y = np.random.rand(50)
hist_data = np.random.randn(1000)
```

Create figure with 3 subplots

```
fig, axes = plt.subplots(1, 3, figsize=(12, 4))
```

Bar chart

```
axes[0].bar(x, y, color='skyblue')
axes[0].set_title("Bar Chart")
```

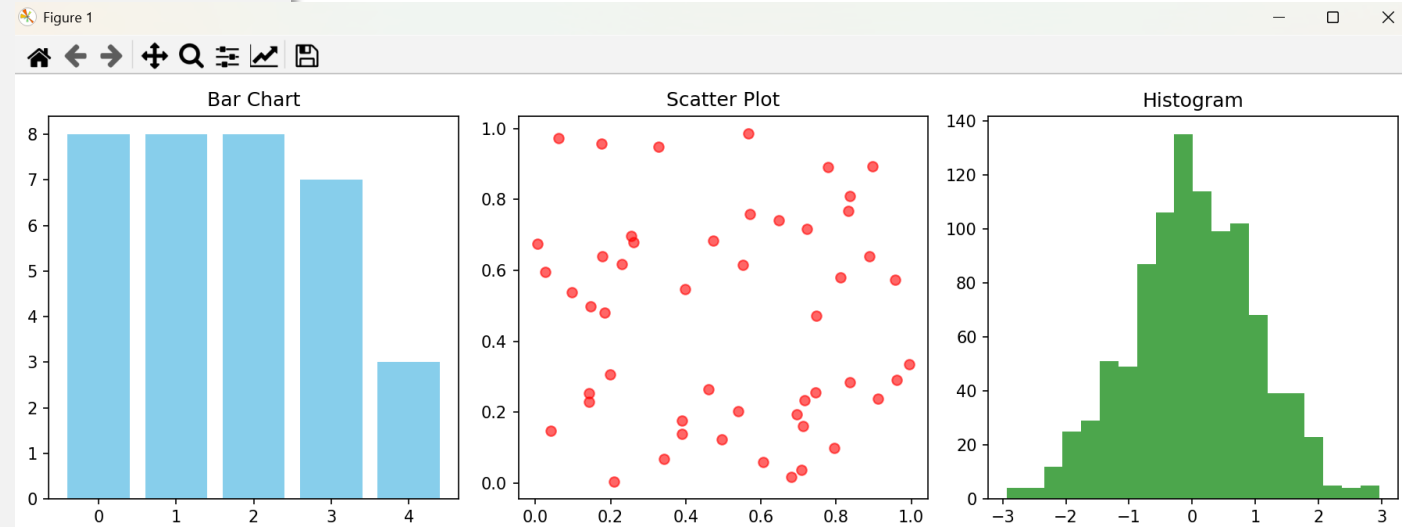
Scatter plot

```
axes[1].scatter(scatter_x, scatter_y, c='red', alpha=0.6)
axes[1].set_title("Scatter Plot")
```

Histogram

```
axes[2].hist(hist_data, bins=20, color='green', alpha=0.7)
axes[2].set_title("Histogram")
```

```
plt.tight_layout()
plt.show()
```



creates a bar chart, scatter plot, and histogram

Thank you!

